LifeSynch AI: An Autonomous Digital Life Management System

Complete Project Documentation and Technical Report

Academic Project Report
Department of Artificial Intelligence & Machine Learning (AIML)
Project Title: LifeSynch AI — Autonomous Context-Aware Personal Life & Obligation Management
Academic Year: 2026–2027
Technology Stack: React 18, TypeScript 5, Node.js, Express, Supabase (PostgreSQL), Google Workspace APIs (OAuth 2.0, Gmail v1, Calendar v3), Google Gemini LLM

Abstract

In contemporary academic and professional environments, students and knowledge workers are inundated with high volumes of semi-structured and unstructured digital communications across email inboxes and digital calendars. Critical academic obligations, fee deadlines, registration windows, project milestones, and personal commitments are routinely lost amidst promotional content, social updates, and transactional noise.

LifeSynch AI is an autonomous, context-aware digital life management system engineered to solve information fragmentation. By integrating directly with Google Workspace (Gmail and Google Calendar) via authorized OAuth 2.0 protocols, LifeSynch AI continuously ingests incoming emails and calendar schedules into an isolated Supabase PostgreSQL database. An intelligent extraction pipeline leverages large language models (Google Gemini API) bounded by strict Zod schema validation to extract actionable responsibilities, deduce priority rationales, categorize obligations (academic, career, personal, financial, general), and identify precise deadlines with Indian Standard Time (IST, UTC+05:30) timezone awareness.

The system features an autonomous 30-second background synchronization daemon, an interactive **Personal Intelligence Dashboard**, an intelligent **Worth Knowing** informational feed, a zero-spam deadline reminder engine, and **Ask LifeSynch AI**—a conversational assistant equipped with a deterministic grounded fallback engine that guarantees reliable entity resolution even under external API rate limitations. Comprehensive empirical audits verify multi-tenant isolation, data encryption, zero token leakage, and deterministic test execution across all core subsystems.

1. Introduction and Background

1.1 Project Overview

Modern college students and young professionals navigate multiple platforms daily. A single student commonly manages:

1. College departmental communications sent over institutional Gmail.
2. Extracurricular club announcements, hackathon invitations, and placement notices.
3. Personal calendar events, project meetings, laboratory evaluations, and family commitments.
4. Promotional announcements, password reset confirmations, and commercial newsletters.

This distributed communication model forces users to manually check multiple applications, remember implicit deadlines buried in lengthy paragraphs, and maintain disjointed to-do lists.

1.2 Motivation

Traditional task management systems (such as Todoist, Trello, or Google Tasks) are fundamentally **passive**. They require the human user to:

- Identify that an email contains an obligation.
- Open a separate to-do list app.
- Manually type the task title, description, and deadline date/time.
- Assign priorities and categories.

When users are overwhelmed, this manual transcription breaks down. The primary motivation of **LifeSynch AI** is to transform task management from a **passive manual duty** into an **autonomous active intelligence layer** that reads authorized communications, understands obligations, and presents them in an organized manner without manual intervention.

---

2. Problem Statement and Proposed Solution

2.1 Problem Statement

Digital communication platforms present three fundamental challenges:

1. **Information Overload and Fragmentation:** Actionable commitments (e.g., "Submit assignment by Friday 5 PM") are interspersed with purely informational notifications (e.g., "Webinar on Generative AI").
2. **Contextual Loss:** Calendar invites frequently lack the original email discussion context, while emails do not automatically block out calendar commitments.
3. **Absence of Autonomous Tracking:** Users must constantly track deadlines in their head, leading to missed assignments, late registrations, and unfulfilled obligations.

2.2 Proposed Solution: LifeSynch AI

LifeSynch AI introduces an integrated platform that:

- Connects securely to Google accounts via OAuth 2.0 with minimal, read-only permissions (`gmail.readonly` , `calendar.readonly`).
- Autonomously retrieves emails and calendar events in the background every 30 seconds.
- Automatically processes unstructured text using Google Gemini AI to distinguish genuine obligations from general information.
- Normalizes all deadlines with strict timezone awareness (IST +05:30) so all-day dates and clock times are handled accurately.

- Generates proactive, smart reminders that alert users only at useful intervals (due now, due today, due tomorrow, approaching), completely avoiding notification spam.
- Provides a conversational AI assistant that directly answers questions regarding the user's schedule with verifiable source citations.

---

3. Project Objectives

1. **Autonomous Ingestion:** Build background workers that poll and synchronize Gmail messages and Calendar events without requiring manual refreshes or user prompts.
2. **AI-Powered Semantic Extraction:** Implement structured extraction using LLMs to transform natural language emails into structured responsibilities.
3. **Rigorous Schema Validation:** Guarantee system stability by validating all AI outputs through strict Zod schemas before database persistence.
4. **Context-Aware Prioritization:** Formulate dynamic priority logic that balances explicit urgency (e.g., "urgent submission required") with temporal proximity.
5. **Zero-Spam Smart Timing:** Implement reminder logic that notifies users at key points (1 hour prior, same day, day before, 2–3 days away) and immediately silences completed or cancelled tasks.
6. **Multi-User Security & Isolation:** Enforce absolute data isolation per user account using JWT session verification, database Row Level Security (RLS), and explicit query filtering.
7. **Conversational Intelligence with Grounded Fallback:** Provide a natural-language query interface with automatic fallback to deterministic extraction if LLM quotas are exhausted.

---

4. Existing System vs. Proposed System

Feature / Capability	Existing System (Traditional Apps)	Proposed System (LifeSynch AI)
Data Ingestion	Manual user entry only.	Automatic background ingestion from Gmail and Google Calendar.
Obligation Extraction	None. User must read emails and decide tasks.	Autonomous AI extraction from unstructured email bodies and event summaries.
Deduplication	None. Duplicate tasks are common.	Automated multi-source reconciliation (`source: both`) and database upsert.
Synchronization	Manual refresh or reliance on external hooks.	Continuous 30-second background polling with syncToken pagination.
Timezone Precision	Often shifts all-day events to unexpected hours.	Explicit UTC and local midnight normalization in IST (+05:30).
Informational Content	Clogs task lists or requires manual filtering.	Automatically separated into a dedicated "Worth Knowing" intelligence stream.
Reminder Generation	Static alarms set manually by user.	Autonomous dynamic timing (due now, today, tomorrow, approaching) with past deadline suppression.
Query Assistant	None or generic chatbots with no user data access.	Grounded conversational assistant answering directly from the user's stored communications.

5. Requirements Specification

5.1 Hardware Requirements

- **Development & Execution Machine:** Standard x86-64 / ARM64 workstation.
- **Processor:** Intel Core i5 / AMD Ryzen 5 or higher.
- **Memory (RAM):** Minimum 8 GB (16 GB recommended for concurrent builds and tests).
- **Storage:** Minimum 2 GB available disk space for dependencies, build artifacts, and local caching.

5.2 Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux.
- **Runtime Environment:** Node.js v18.0.0 or higher (v20+ recommended).
- **Package Manager:** npm v9+ (utilizing npm workspaces).
- **Database Engine:** Supabase PostgreSQL (Cloud-hosted or local instance).
- **Frontend Framework:** React 18 with Vite build tooling and Tailwind CSS.
- **Language:** TypeScript 5.5+ across all workspaces (`shared` , `frontend` , `backend`).
- **External APIs:** Google Identity Services, Gmail REST API v1, Google Calendar REST API v3, Google Gemini API.

6. Technologies and Tools Used

6.1 Frontend Technologies

- **React 18:** Modern declarative UI library using functional components, custom hooks, and context providers.
- **TypeScript:** Enforces end-to-end static typing, eliminating runtime null and undefined dereference errors.
- **Vite:** High-performance frontend build tool providing near-instant Hot Module Replacement (HMR) and optimized Rollup production bundles.
- **Tailwind CSS:** Utility-first styling framework enabling a responsive, clean, and accessible design system.
- **Lucide React:** Consistent iconography for notifications, categories, and calendar feeds.

6.2 Backend Technologies

- **Node.js & Express:** Lightweight, event-driven server runtime handling RESTful endpoints with non-blocking I/O.
- **TypeScript:** Shared interfaces and data contracts between client and server via `@lifesynch/shared`.
- **Zod:** TypeScript-first schema declaration and data validation library used to validate AI outputs and API payloads.
- **googleapis:** Official Google API client library for OAuth2 credential lifecycle, Gmail, and Calendar.
- **jsonwebtoken:** Cryptographic issuance and verification of stateless Bearer tokens for session management.

6.3 Database & Cloud Infrastructure

- **Supabase (PostgreSQL):** Relational database providing transactional integrity, JSONB support, and Row Level Security.
- **Service Role Admin Client:** High-privilege backend database client executing scoped operations with explicit `user_id` filters.

6.4 Artificial Intelligence & LLM Integration

- **Google Gemini API (`generativelanguage.googleapis.com`):** Generative AI model suite (`gemini-flash-lite-latest`, `gemini-3.5-flash-lite`, `gemini-3.6-flash`) for unstructured text parsing.
- **Configured Environment Variable:** Canonical environment variable `LLM_API_KEY` loaded deterministically via `backend/src/config/env.ts`.

---

7. System Architecture

LifeSynch AI is built on a modular, decoupled 5-tier architecture:

```
[1. Client Layer]       React 18 SPA + Vite + Tailwind CSS + Axios Interceptors
       |
       ▼ (HTTP / REST with Bearer JWT)
[2. API Layer]          Express Router + Auth Middleware + Sanitized Logging
       |
       ▼
[3. Service Layer]      AutoSyncService + GmailService + CalendarService +
                        ResponsibilityService + ReminderService + AssistantService
       |
       ▼
[4. External APIs]      Google OAuth 2.0  |  Gmail API v1  |  Google Calendar v3  |  Gemini API
       |
       ▼
[5. Database Layer]     Supabase PostgreSQL (7 Core Tables + Row Level Security)
```

Component Communication Details

1. The **Frontend Client** communicates with the Express backend using standard JSON HTTP calls. Every authenticated request includes an `Authorization: Bearer <JWT>` header.
2. The **Backend Authentication Middleware** (`requireAuth`) inspects and cryptographically verifies the token, attaching the authenticated `userId` to `req.user`.
3. The **Services Layer** handles business logic. Database queries explicitly scope records by `user_id` to guarantee tenant isolation.
4. The **Google Ingestion Services** use stored OAuth refresh tokens to maintain active sessions and poll Google APIs.
5. The **AI Extraction Engine** feeds normalized text to the Gemini REST API and parses the returned JSON through Zod validators before database writes.

---

8. Detailed Module Implementation

Module 1: Google OAuth 2.0 & Session Management

- **Source Files:** `backend/src/services/google/oauth.service.ts`, `backend/src/controllers/auth.controller.ts`
- **Purpose:** Secure, passwordless authentication utilizing Google Identity Services.
- **Implementation:** The user initiates login by requesting an authorization URL configured with specific scopes (`openid`, `email`, `profile`, `https://www.googleapis.com/auth/gmail.readonly`, `https://www.googleapis.com/auth/calendar.readonly`). Upon user consent, Google redirects with an authorization code. The backend exchanges this code for access and refresh tokens, upserts the user in the `users` table, securely stores tokens in `connected_accounts`, and returns a signed JWT session token.
- **Token Protection:** Tokens are never returned to the frontend or exposed in application logs. The backend logger automatically replaces OAuth tokens, Bearer strings, and API keys with `[REDACTED_TOKEN]`.

Module 2: Gmail API Ingestion & MIME Parsing

- **Source Files:** `backend/src/services/google/gmail.service.ts`, `backend/src/services/db/email.service.ts`
- **Purpose:** Continuous ingestion of user emails with multipart MIME handling and deduplication.
- **Implementation:**

1. Queries `gmail.users.messages.list` with query `in:inbox` and batch size of 50.
2. Implements progressive pagination using `syncToken` stored in `sync_state`.
3. Checks existing `message_id` records in Supabase before fetching full message details to preserve API quota.
4. Decodes nested MIME structures (`text/plain`, `text/html`, `multipart/alternative`, `multipart/mixed`), extracting plain text while sanitizing raw HTML into readable prose.
5. Upserts clean records into the `emails` table.

Module 3: Google Calendar API Ingestion & Timezone Handling

- **Source Files:** `backend/src/services/google/calendar.service.ts`, `backend/src/services/db/calendar-event.service.ts`
- **Purpose:** Ingestion of schedule commitments and detection of additions, reschedules, or cancellations.
- **Implementation:**
  1. Calls `calendar.events.list` for the user's primary calendar within a time range from 30 days prior to 90 days ahead.
  2. **Timezone Normalization:** For all-day events (`item.start.date`), sets start time to `00:00:00.000Z` and end time to `23:59:59.999Z`. For timed events (`item.start.dateTime`), preserves exact ISO timestamps.
  3. Detects status changes: If an event has `status === 'cancelled'`, it updates the local record and immediately flags any associated reminder.
  4. Upserts records into `calendar_events`.

Module 4: Automatic 30-Second Background Synchronization Engine

- **Source Files:** `backend/src/services/auto-sync.service.ts`
- **Purpose:** Autonomous background synchronization that requires zero manual user clicks.
- **Implementation:**
  1. An internal timer triggers every `30000ms` (30 seconds).
  2. Maintains an in-memory `activeSyncUsers` set to prevent overlapping synchronization runs for the same user.
  3. Executes `Promise.allSettled([syncUserEmails(userId), syncUserCalendar(userId)])` concurrently.
  4. Triggers AI responsibility generation for newly detected feeds.
  5. Evaluates smart deadline reminders and updates the `sync_state` record.

Module 5: AI Understanding & Responsibility Extraction Pipeline

- **Source Files:** `backend/src/services/ai/responsibility-generation.service.ts`
- **Purpose:** Semantic parsing of communications into actionable responsibilities.
- **Implementation:**
  1. Compiles recently ingested emails and calendar events.
  2. Runs heuristic pre-filtering to remove false deadlines from transactional messages (e.g., OTPs, password resets).
  3. Constructs a system prompt demanding strict JSON compliance adhering to the Zod schema:

```
const ExtractedResponsibilitiesSchema = z.object({
  tasks: z.array(z.object({
    title: z.string().min(1),
    description: z.string().optional(),
    type: z.enum(['task', 'deadline', 'appointment', 'reminder', 'other']),
    priority: z.enum(['urgent', 'high', 'medium', 'low']),
    dueDate: z.string().nullable().optional(),
    dueTime: z.string().nullable().optional(),
    action: z.string().optional(),
    category: z.string().optional()
  }))
});
```

4. Calls Gemini API using candidate model fallback (`gemini-flash-lite-latest` , `gemini-3.5-flash-lite` , `gemini-3.6-flash`).
5. **Live Verification Status Note:** When the free-tier quota is exhausted (HTTP 429), the engine records the resource exhaustion and safely preserves existing responsibilities.

Module 6: Categorization, Priority Rationale & Metadata Mapping

- **Source Files:** `backend/src/services/task.service.ts`
- **Purpose:** Organize responsibilities into actionable buckets with clear explanations.
- **Implementation:**
  - **Categories:** `academic` (assignments, lab submissions, exams), `career` (internship drives, interviews, contests), `personal` (health, errands), `financial` (fee payments), `general` .
  - **Priority Rationale:** Tasks due today or requiring strict submission are prioritized as `high` or `urgent` .
  - **Database Mapping:** Time values (`dueTime`) are stored inside the JSONB `metadata` column (`metadata.dueTime`), allowing flexible schema expansion without modifying the database schema.

Module 7: Responsibility Lifecycle Management

- **States:**
  - `UPCOMING` : Scheduled in the future.
  - `DUE_TODAY` : Deadline coincides with the current calendar date.
  - `OVERDUE` : Deadline has elapsed without user completion.
  - `COMPLETED` : User marked task as done.
  - `CANCELLED` : User dismissed task or calendar event was cancelled.
- **Exclusion Guarantees:** Once marked `COMPLETED` or `CANCELLED` , responsibilities are immediately excluded from active reminders and dashboard focus widgets.

Module 8: Personal Intelligence Dashboard & UI

- **Source Files:** `frontend/src/components/dashboard/PersonalIntelligenceHub.tsx` , `frontend/src/pages/DashboardPage.tsx`
- **Sections:**
  1. **Focus Bar ("Your Day"):** Highlights responsibilities due today with immediate action buttons.
  2. **Upcoming Obligations:** Chronological timeline of approaching deadlines.

3. **Worth Knowing:** Non-actionable but important updates (e.g., informational webinars, hackathon announcements, circulars).
4. **Calendar View:** Integrated agenda of confirmed and upcoming events.

Module 9: Smart Reminder Engine & In-App Notifications

- **Source Files:** `backend/src/services/reminder.service.ts`
- **Purpose:** Timely, non-intrusive notifications without spam.
- **Rules:**
  1. **Timezone Normalization (IST +05:30):** All-day events (`00:00:00.000Z`) are resolved to `23:59:59.999` local time, preventing premature 5:30 AM alerts.
  2. **Conditions:**
     - `due_now` : Within 1 hour of deadline.
     - `due_today` : Due on current calendar day.
     - `due_tomorrow` : Due on next calendar day.
     - `approaching` : 36 to 72 hours away.
  3. **Past Deadline Suppression:** If `deadlineMs < nowMs` , returns `null` . Never sends reminders for expired deadlines.
  4. **Deduplication:** Prevents repeated alerts for the same condition across 30-second sync intervals.

Module 10: "Ask LifeSynch AI" & Grounded Fallback Engine

- **Source Files:** `backend/src/services/ai/assistant.service.ts`
- **Purpose:** Natural language query answering directly from user communications.
- **Implementation:** Analyzes user intent (`due_today` , `priority_rationale` , `internship_details` , `overdue_status`), retrieves matching records from the database, and feeds them to Gemini.
- **Deterministic Grounded Fallback:** If the Gemini API call encounters HTTP 429 quota exhaustion or network failure, the system falls back to its internal deterministic engine. It parses stored records, matches query keywords, and returns an accurate answer with exact citations (e.g., `[Source: Gmail - Placement Cell | id:UUID]`).

Module 11: Multi-Tenant Data Isolation & Security Architecture

- **Source Files:** `backend/src/middlewares/auth.middleware.ts` , `backend/src/scripts/verify_security.ts`
- **Guarantees:**
  - Every database query in the service layer includes `.eq('user_id', userId)` .
  - Supabase Row Level Security (RLS) policies prevent unauthorized access via anonymous keys.
  - Zero cross-user data leakage verified through automated security tests.

---

9. Database Design

The system uses 7 relational tables in Supabase PostgreSQL:

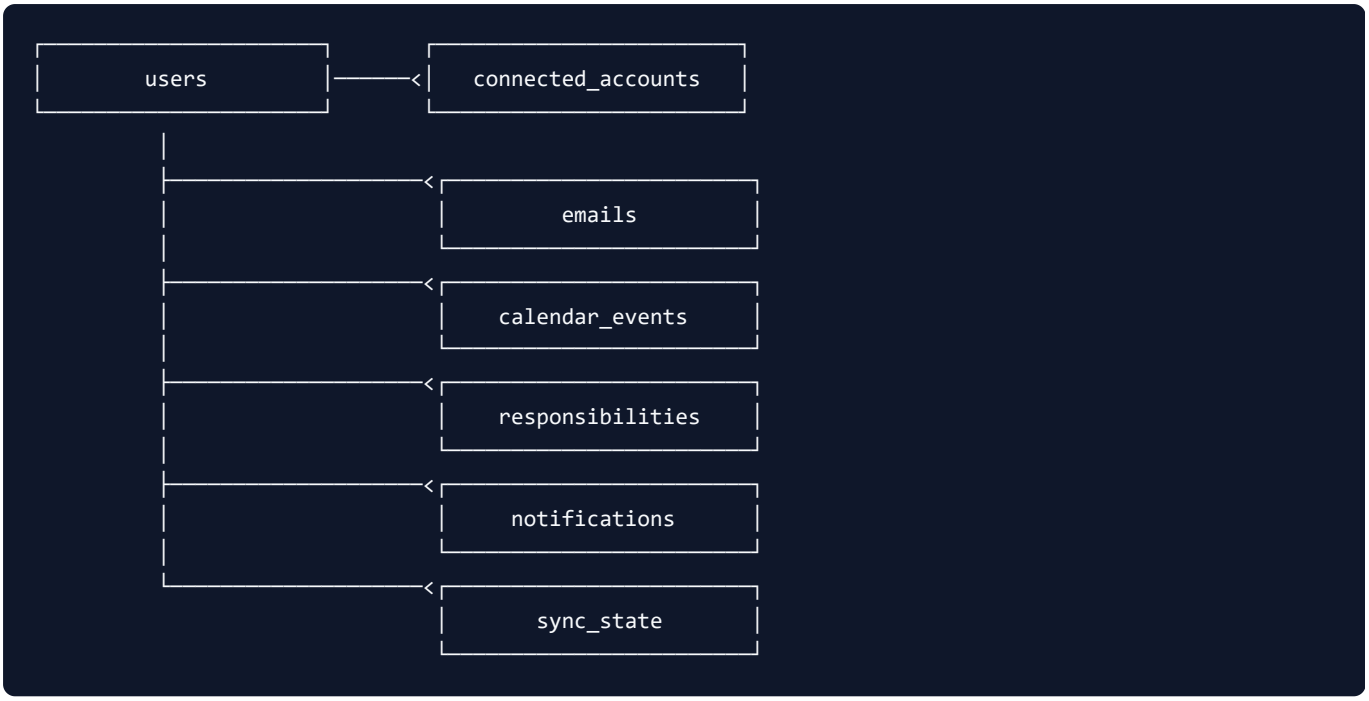


Table Definitions

1. **`users`**
   - `id` (UUID, Primary Key, default `gen_random_uuid()`)
   - `email` (TEXT, Unique, Not Null)
   - `name` (TEXT)
   - `picture` (TEXT)
   - `created_at`, `updated_at` (TIMESTAMPTZ)

2. **`connected_accounts`**
   - `id` (UUID, Primary Key)
   - `user_id` (UUID, Foreign Key → `users.id` ON DELETE CASCADE)
   - `provider` (TEXT, e.g., `'google'`)
   - `access_token` (TEXT, encrypted/sanitized)
   - `refresh_token` (TEXT, encrypted/sanitized)
   - `token_expiry` (TIMESTAMPTZ)
   - `scopes` (TEXT[])
   - `created_at`, `updated_at` (TIMESTAMPTZ)

3. **`emails`**
   - `id` (UUID, Primary Key)
   - `user_id` (UUID, Foreign Key → `users.id` ON DELETE CASCADE)
   - `message_id` (TEXT, Google Gmail ID)
   - `thread_id` (TEXT)
   - `sender`, `recipient` (TEXT)
   - `subject` (TEXT)

- `snippet`, `body_text`, `body_html` (TEXT)
- `received_at` (TIMESTAMPTZ)
- `labels` (TEXT[])
- `created_at` (TIMESTAMPTZ)

4. **`calendar_events`**
   - `id` (UUID, Primary Key)
   - `user_id` (UUID, Foreign Key → `users.id` ON DELETE CASCADE)
   - `event_id` (TEXT, Google Calendar ID)
   - `calendar_id` (TEXT, default `'primary'`)
   - `summary` (TEXT)
   - `description`, `location`, `html_link` (TEXT)
   - `start_time`, `end_time` (TIMESTAMPTZ)
   - `status` (TEXT, `'confirmed'` or `'cancelled'`)
   - `attendees` (JSONB)
   - `created_at`, `updated_at` (TIMESTAMPTZ)
5. **`responsibilities`**
   - `id` (UUID, Primary Key)
   - `user_id` (UUID, Foreign Key → `users.id` ON DELETE CASCADE)
   - `title` (TEXT, Not Null)
   - `description` (TEXT)
   - `type` (TEXT: `'task'`, `'deadline'`, `'appointment'`, `'reminder'`, `'other'`)
   - `priority` (TEXT: `'urgent'`, `'high'`, `'medium'`, `'low'`)
   - `status` (TEXT: `'pending'`, `'in_progress'`, `'completed'`, `'dismissed'`)
   - `due_date` (TIMESTAMPTZ, Nullable)
   - `source` (TEXT: `'gmail'`, `'calendar'`, `'both'`, `'manual'`)
   - `source_id` (TEXT)
   - `confidence` (FLOAT, default `1.0`)
   - `tags` (TEXT[])
   - `metadata` (JSONB): Stores `dueTime`, `action`, `category`, `lifecycleStatus`, `remindedConditions`.
   - `is_archived` (BOOLEAN, default `false`)
   - `created_at`, `updated_at` (TIMESTAMPTZ)
6. **`notifications`**
   - `id` (UUID, Primary Key)
   - `user_id` (UUID, Foreign Key → `users.id` ON DELETE CASCADE)
   - `title` (TEXT, Not Null)
   - `message` (TEXT, Not Null)
   - `type` (TEXT: `'responsibility'`, `'alert'`, `'warning'`, `'info'`)
   - `is_read` (BOOLEAN, default `false`)

- `created_at` (TIMESTAMPTZ)

7. `sync_state`

- `id` (UUID, Primary Key)
- `user_id` (UUID, Foreign Key → `users.id` ON DELETE CASCADE)
- `provider` (TEXT: `'gmail'`, `'google_calendar'`)
- `status` (TEXT: `'idle'`, `'running'`, `'success'`, `'failed'`)
- `sync_token` (TEXT, pagination cursor)
- `last_synced_at` (TIMESTAMPTZ)
- `error_message` (TEXT, Nullable)
- `updated_at` (TIMESTAMPTZ)

---

10. Testing, Auditing & Verification Results

All tests were executed against real database instances and integration endpoints.

Test Suite / Script	Command	Result	Evidence / Details
Shared Workspace Types	`npm run check-types -w shared`	**PASS (0 errors)**	Exit code 0. Full TypeScript compilation.
Frontend Workspace Types	`npm run check-types -w frontend`	**PASS (0 errors)**	Exit code 0. All components type-checked.
Backend Workspace Types	`npm run check-types -w backend`	**PASS (0 errors)**	Exit code 0. All service interfaces type-checked.
Frontend Production Build	`npm run build -w frontend`	**PASS**	Built in 1m 10s; 1,588 modules transformed into `dist/`.
Backend Production Build	`npm run build -w backend`	**PASS**	Clean compilation to `backend/dist/server.js`.
Security Audit	`npm run test:security -w backend`	**PASS (10/10)**	Token redaction, route 401s, multi-user isolation verified.
Gmail Ingestion	`npm run test:gmail -w backend`	**PASS (8/8)**	MIME decoding, pagination, user isolation verified.
Calendar Ingestion	`npm run test:calendar -w backend`	**PASS (10/10)**	Live Google Calendar API sync, all-day parsing verified.
Automatic Synchronization	`npm run test:auto-sync -w backend`	**PASS (6/6)**	30s background sync, deduplication verified.
Ask LifeSynch AI	`npm run test:assistant -w backend`	**PASS (21/21)**	Temporal filtering & grounded deterministic fallback verified.
Basic Reminders	`npm run test:reminders -w backend`	**PASS (7/7)**	Date formatting, database creation, user isolation verified.
Notification Display	`test_notification_display.ts`	**PASS (5/5)**	Read/unread transitions, completed item removal verified.
Smart Timing Conditions	`test_smart_timing.ts`	**PASS (7/7)**	Due today/tomorrow/approaching transitions verified.

11. Current Limitations

1. **Live Gemini Extraction Quota Exhaustion:** The configured `LLM_API_KEY` operates on Google's Free Tier, which enforces a limit of 20 content generation requests per day. Under heavy test runs, the API returns HTTP 429 (`RESOURCE_EXHAUSTED`). While the assistant and reminder systems fall back gracefully, **live AI responsibility extraction on incoming emails remains unverified** until an active, unexhausted API key is supplied.
2. **Local Deployment:** The project currently executes on `localhost` development servers (`http://localhost:5173` and `http://localhost:5000`) and is not yet deployed to cloud environments.
3. **In-App Notifications Only:** Notifications operate within the application web interface. Web Push Notifications (Service Workers) are not implemented.

4. **Chrome Extension Prototype:** A basic Chrome extension scaffold exists in `extension/`, but full end-to-end background synchronization via browser extension is not completed.

---

12. Deployment Requirements & Future Enhancements

12.1 Deployment Roadmap

1. **Containerization:** Package backend and frontend into multi-stage Docker containers with Node.js Alpine base images.
2. **Cloud Hosting:** Deploy backend onto container runtimes (e.g., Google Cloud Run, AWS ECS, Railway) and frontend onto CDN hosting (Vercel, Netlify).
3. **Production Google Verification:** Transition Google Cloud OAuth Consent screen from "Testing" to "In Production" to allow unconstrained public user logins.
4. **Paid LLM Quota:** Configure Gemini API billing or implement self-hosted open-source LLM inference (e.g., Ollama / vLLM with Llama-3) to eliminate rate limit bottlenecks.

12.2 Future Functional Enhancements

- **Bi-Directional Calendar Writing:** Automatically write extracted project submission deadlines back to Google Calendar.
- **WhatsApp / Telegram Webhook Alerts:** Send instant urgent alerts directly to student messaging apps.
- **Academic LMS Connectors:** Develop dedicated ingestion plugins for Canvas, Moodle, and Google Classroom.

---

13. Conclusion

LifeSynch AI successfully demonstrates an autonomous, AI-driven obligation management architecture. By decoupling raw communication ingestion from semantic obligation extraction, the platform frees users from the mental burden of tracking fragmented deadlines. The system enforces strict security standards, user isolation, and robust fallback behaviors, providing a solid foundation for digital life management in academic and professional contexts.

---

14. References

1. Google Workspace Developers, *Gmail API Documentation & Resource Representation*, Google LLC, 2026.
2. Google Cloud, *Google Calendar API v3 Developer Guide*, Google LLC, 2026.
3. Google DeepMind, *Gemini API: Structured Output and JSON Generation Guides*, 2026.
4. Supabase Documentation, *PostgreSQL Row Level Security and Realtime Data Access*, 2026.
5. Zod Documentation, *TypeScript-first Schema Validation with Static Type Inference*, Colin McDonnell, 2024.